

Rubaisha Arif
CT-25067
Section B

Object Oriented Programming (CT-260)
Lab 08

Example of Diamond Problem:

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 class A{ 4 public: 5 int a; 6 }; 7 class B : virtual public A{ 8 public: 9 int b; 10 }; 11 class C : virtual public A{ 12 public: 13 int c; 14 }; 15 class D : public B, public C{ 16 public: 17 int d; 18 }; 19 int main(){ 20 D obj; 21 obj.a = 200; 22 cout<<obj.a; 23 return 0;}</pre>	<pre>200 === Code Execution Successful ===</pre>

EXAMPLE 2

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 class PoweredDevice { 4 public: 5 PoweredDevice(int power) { 6 cout << "PoweredDevice: " << power << '\n'; 7 } 8 }; 9 class Scanner : public PoweredDevice { 10 public: 11 Scanner(int scanner, int power) : PoweredDevice(power) { 12 cout << "Scanner: " << scanner << '\n'; 13 } 14 }; 15 16 class Printer : public PoweredDevice { 17 public: 18 Printer(int printer, int power) : PoweredDevice(power) { 19 cout << "Printer: " << printer << '\n'; 20 } 21 }; 22 class Copier : public Scanner, public Printer { 23 public: 24 Copier(int scanner, int printer, int power) 25 : Scanner(scanner, power), Printer(printer, power) { 26 } 27 }; 28 int main() { 29 Copier copier(1, 2, 3); 30 return 0; 31 }</pre>	<pre>PoweredDevice: 3 Scanner: 1 PoweredDevice: 3 Printer: 2 === Code Execution Successful ===</pre>

EXAMPLE 3(By using Virtual Base Classes)

main.cpp	Run	Output
<pre>1 #include <iostream> 2 using namespace std; 3 class PoweredDevice { 4 public: 5 PoweredDevice(int power) { 6 cout << "PoweredDevice: " << power << '\n'; } 7 }; 8 9 class Scanner : virtual public PoweredDevice { 10 public: 11 Scanner(int scanner, int power) : PoweredDevice(power) { 12 cout << "Scanner: " << scanner << '\n'; } 13 }; 14 15 class Printer : virtual public PoweredDevice { 16 public: 17 Printer(int printer, int power) : PoweredDevice(power) { 18 cout << "Printer: " << printer << '\n'; 19 } 20 }; 21 22 class Copier : public Scanner, public Printer { 23 public: 24 Copier(int scanner, int printer, int power) 25 : PoweredDevice(power), Scanner(scanner, power), Printer(printer, power) {} 26 }; 27 28 int main() { 29 Copier copier(1, 2, 3); 30 return 0; 31 }</pre>		<pre>PoweredDevice: 3 Scanner: 1 Printer: 2 === Code Execution Successful ===</pre>

EXERCISE

Q1

main.cpp	Run	Output
<pre>1 #include <iostream> 2 #include <string> 3 using namespace std; 4 class Vehicle { 5 protected: 6 string TypeOfCar, Make, Model, Color; 7 int Year, MilesDriven; 8 public: 9 Vehicle(string toc, string mk, string ml, string c, int y, int md) 10 : TypeOfCar(toc), Make(mk), Model(ml), Color(c), Year(y), MilesDriven(md) {} 11 }; 12 13 class GasVehicle : virtual public Vehicle { 14 protected: 15 float FuelTankSize; 16 public: 17 GasVehicle(string toc, string mk, string ml, string c, int y, int md, float fts) 18 : Vehicle(toc, mk, ml, c, y, md), FuelTankSize(fts) {} 19 }; 20 21 22 class ElectricVehicle : virtual public Vehicle { 23 protected: 24 float EnergyStorage; 25 public: 26 ElectricVehicle(string toc, string mk, string ml, string c, int y, int md, float es) 27 : Vehicle(toc, mk, ml, c, y, md), EnergyStorage(es) {} 28 }; 29 30 class HighPerformance : public GasVehicle { 31 protected: 32 int HorsePower; 33 float Speed; 34 public: 35 HighPerformance(string toc, string mk, string ml, string c, int y, int md, float fts, int hp, float s) 36 : Vehicle(toc, mk, ml, c, y, md), GasVehicle(toc, mk, ml, c, y, md, fts), HorsePower(hp), Speed(s) {} 37 }; 38 39 class HeavyVehicle : public GasVehicle, public ElectricVehicle { 40 protected: 41 float MaximumWeight, Length; 42 int NumberOfWheels; 43 public: 44 HeavyVehicle(string toc, string mk, string ml, string c, int y, int md, float fts, float es, float mw, float l, int now) 45 : Vehicle(toc, mk, ml, c, y, md), GasVehicle(toc, mk, ml, c, y, md, fts), 46 ElectricVehicle(toc, mk, ml, c, y, md, es), MaximumWeight(mw), Length(l), NumberOfWheels(now) {}</pre>		<pre>-----DETAILS----- VEHICLE Type of Car: SUV Make : Hino Model : Kazay Color : Black Year : 2022 Miles Driven : 1550 GAS VEHICLE Fuel Tank Size : 250.5 Electric Vehicle Energy Storage : 110 Heavy Vehicle Max Weight : 18000 Length : 11.5 No. of Wheels : 4 Bus No. of Seats : 6 === Code Execution Successful ===</pre>

```

38
39- class HeavyVehicle : public GasVehicle, public ElectricVehicle {
40- protected:
41-     float MaximumWeight, Length;
42-     int NumberOfWheels;
43- public:
44-     HeavyVehicle(string toc, string mk, string ml, string c, int y, int md, float fts, float es, float mw, float l, int now)
45-         : Vehicle(toc, mk, ml, c, y, md), GasVehicle(toc, mk, ml, c, y, md, fts),
46-         ElectricVehicle(toc, mk, ml, c, y, md, es), MaximumWeight(mw), Length(l), NumberOfWheels(now) {}
47- };
48
49- class SportsCar : public HighPerformance {
50-     string GearBox, DriveSystem;
51- public:
52-     SportsCar(string toc, string mk, string ml, string c, int y, int md, float fts, int hp, float s, string gb, string ds)
53-         : Vehicle(toc, mk, ml, c, y, md), HighPerformance(toc, mk, ml, c, y, md, fts, hp, s), GearBox(gb), DriveSystem(ds) {}
54- };
55
56- class ConstructionTruck : public HeavyVehicle {
57-     string Cargo;
58- public:
59-     ConstructionTruck(string toc, string mk, string ml, string c, int y, int md, float fts, float es, float mw, float l, int now
60-         , string crg)
61-         : Vehicle(toc, mk, ml, c, y, md), HeavyVehicle(toc, mk, ml, c, y, md, fts, es, mw, l, now), Cargo(crg) {}
62- };
63
64- class Bus : public HeavyVehicle {
65-     int NumberOfSeats;
66- public:
67-     Bus(string toc, string mk, string ml, string c, int y, int md, float fts, float es, float mw, float l, int now, int nos)
68-         : Vehicle(toc, mk, ml, c, y, md), HeavyVehicle(toc, mk, ml, c, y, md, fts, es, mw, l, now), NumberOfSeats(nos) {}
69-
70-     void display() {
71-         cout << "-----DETAILS-----\n\n";
72-         cout << "Vehicle\n" << endl;
73-         cout << "Type of Car: " << TypeOfCar << endl;
74-         cout << "Make : " << Make << endl;
75-         cout << "Model : " << Model << endl;
76-         cout << "Color : " << Color << endl;
77-         cout << "Year : " << Year << endl;
78-         cout << "Miles Driven : " << MilesDriven << endl;
79-         cout << "Gas Vehicle\n" << endl;
80-         cout << "Fuel Tank Size : " << FuelTankSize << endl;
81-         cout << "Electric Vehicle\n" << endl;
82-         cout << "Energy Storage : " << EnergyStorage << endl;
83-         cout << "Heavy Vehicle\n" << endl;
84-         cout << "Max Weight : " << MaximumWeight << endl;
85-         cout << "Length : " << Length << endl;
86-         cout << "No. of Wheels : " << NumberOfWheels << endl;
87-         cout << "Bus\n" << endl;
88-         cout << "No. of Seats : " << NumberOfSeats << endl; }
89- };
90- int main(){
91-     Bus b("SUV", "Hino", "Kazay", "Black", 2022, 1550, 250.5, 110.0, 18000.0, 11.5, 4, 6);
92-     b.display();
93- }

```

VEHICLE
Type of Car: SUV
Make : Hino
Model : Kazay
Color : Black
Year : 2022
Miles Driven : 1550
GAS VEHICLE
Fuel Tank Size : 250.5
Electric Vehicle
Energy Storage : 110
Heavy Vehicle
Max Weight : 18000
Length : 11.5
No. of Wheels : 4
Bus
No. of Seats : 6
=== Code Execution Successful ===

EXERCISE Q2

```

main.cpp
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 class Character {
5- protected:
6-     string name;
7-     int level, health;
8- public:
9-     Character(string n,int l, int h) : name(n), level(l), health(h) {}
10- };
11- class Warrior : virtual public Character {
12- protected:
13-     int strength;
14-     float melee;
15- public:
16-     Warrior(string n, int l, int h, int st, float m)
17-         : Character(n, l, h), strength(st), melee(m) {}
18-
19-     void slash() {
20-         cout << name << " performs a powerful melee Slash!" << endl; }
21- };
22- class Mage : virtual public Character {
23- protected:
24-     int intelligence;
25-     float spell; //spell casting proficiency
26- public:
27-     Mage(string n, int l, int h, int i, float s)
28-         : Character(n, l, h), intelligence(i), spell(s) {}
29-
30-     void fireball() {
31-         cout << name << " casts a blazing Fireball!" << endl; }
32- };
33- class Archer : virtual public Character {
34- protected:
35-     int dexterity;
36-     float range; //rangw weapons proficiency
37- };

```

Alexa performs a powerful melee Slash!
Alexa casts a blazing Fireball!
=== Code Execution Successful ===

main.cpp

Share

Run

Output

```

30- void fireball() {
31-     cout << name << " casts a blazing Fireball!" << endl; }
32- };
33- class Archer : virtual public Character {
34- protected:
35-     int dexterity;
36-     float range; //rangw weapons proficiency
37- public:
38-     Archer(string n, int l, int h, int d, float r)
39-         : Character(n, l, h), dexterity(d), range(r) {}
40-     void rapidShot() {
41-         cout << name << " fires a Rapid Shot!" << endl; }
42- };
43- class NPC : public Character {
44-     string movementPattern, dialogue;
45- public:
46-     NPC(string n, int l, int h, string p, string d)
47-         : Character(n, l, h), movementPattern(p), dialogue(d) {}
48-     void scriptedDialogue() {
49-         cout << name << " (NPC): " << dialogue << endl; }
50- };
51- class Mighty : public Warrior, public Mage {
52- public:
53-     Mighty(string n, int l, int h, int st, float m, int i, float s)
54-         : Character(n, l, h),
55-           Warrior(n, l, h, st, m),
56-           Mage(n, l, h, i, s)
57-     {}
58-     void showPowers() {
59-         slash();
60-         fireball(); }
61- };
62- int main() {
63-     Mighty player("Alexa", 10, 222, 45, 0.75, 50, 0.98);
64-     player.showPowers();
65-     return 0; }

```

Alexa performs a powerful melee Slash!
Alexa casts a blazing Fireball!

=== Code Execution Successful ===

EXERCISE Q3

main.cpp

Share

Run

Output

```

1  #include <iostream>
2  using namespace std;
3  class InterestAccount;
4  class ChargingAccount;
5  class Account;
6- class ACI {
7  protected:
8      float balance;
9  public:
10-     ACI() {
11-         cout << "Enter initial balance: ";
12-         cin >> balance;
13-     }
14-     ACI(float b) : balance(b) {}
15-     void transfer(float amount, Account& ob);
16-     void transfer(float amount, InterestAccount& ob);
17-     void transfer(float amount, ChargingAccount& ob);
18-     virtual void deposit(float amount) { balance += amount; }
19-     virtual void withdraw(float amount) { balance -= amount; }
20-     void checkBalance() { cout << "Balance: Rs. " << balance << endl; }
21- };
22- class InterestAccount : virtual public ACI {
23- protected:
24-     float interest;
25- public:
26-     InterestAccount() : ACI() { interest = 0.30f; }
27-     InterestAccount(float b, float i) : ACI(b), interest(i) {}
28-     void deposit(float amount) override {
29-         balance += amount + (amount * interest);
30-     }
31- };
32- class ChargingAccount : virtual public ACI {
33- protected:
34-     float fee;
35- public:
36-     ChargingAccount() : ACI() { fee = 25.0f; }

```

Enter initial balance: 3455
Enter initial balance: 444
Balance: Rs. 1744
Balance: Rs. 3355
Balance: Rs. 1874

=== Code Execution Successful ===

main.cpp

Share

Run

Output

```

36     ChargingAccount() : ACI() { fee = 25.0f; }
37     ChargingAccount(float b, float f) : ACI(b), fee(f) {}
38     void withdraw(float amount) override {
39         balance -= (amount + fee);
40     }
41 };
42 class Account : public InterestAccount, public ChargingAccount {
43 public:
44     Account() : ACI() {}
45     Account(float b) : ACI(b) {}
46 };
47 void ACI::transfer(float amount, InterestAccount &ob) {
48     if (balance >= amount) {
49         this->withdraw(amount);
50         ob.deposit(amount);
51     }
52 }
53 void ACI::transfer(float amount, ChargingAccount &ob) {
54     if (balance >= amount) {
55         this->withdraw(amount);
56         ob.deposit(amount);
57     }
58 void ACI::transfer(float amount, Account &ob) {
59     if (balance >= amount) {
60         this->withdraw(amount);
61         ob.deposit(amount);
62     }
63 }
64 int main() {
65     ACI mainAcc;
66     InterestAccount intAcc;
67     intAcc.deposit(1000);
68     intAcc.checkBalance();
69     mainAcc.transfer(100, intAcc);
70     mainAcc.checkBalance();
71     intAcc.checkBalance();
72     return 0;
73 }

```

Enter initial balance: 3455
Enter initial balance: 444
Balance: Rs. 1744
Balance: Rs. 3355
Balance: Rs. 1874

=== Code Execution Successful ===

EXERCISE Q4

main.cpp

Share

Run

Output

```

1 #include <iostream>
2 #include <cstring>
3 using namespace std;
4 class Media {
5 protected:
6     string title;
7     bool borrow;
8 public:
9     Media(string t) : title(t), borrow(false) {}
10    void borrowMedia() {
11        borrow = true;
12        cout << "Borrowed: " << title << endl;
13    }
14    void returnMedia() {
15        borrow = false;
16        cout << "Returned: " << title << endl;
17    }
18    virtual void displayInfo() = 0;
19 };
20
21 class Book : public Media {
22 protected:
23     string author;
24 public:
25     Book(string t, string a) : Media(t), author(a) {}
26     void displayInfo() override {
27         cout << "Book: " << title << ", Author: " << author << endl;
28     }
29 };
30
31 class Magazine : public Media {
32 protected:
33     int issueNo;
34 public:
35     Magazine(string t, int i) : Media(t), issueNo(i) {}
36     void displayInfo() override {

```

Book: Harry Potter, Author: J.K. Rowling
Borrowed: Harry Potter
Magazine: Jung, Issue: 12345
DVD: Inception, Director: Albert
Borrowed: Inception
Returned: Inception

=== Code Execution Successful ===

main.cpp

Share

Run

```
30
31- class Magazine : public Media {
32- protected:
33-     int issueNo;
34- public:
35-     Magazine(string t, int i) : Media(t), issueNo(i) {}
36- void displayInfo() override {
37-     cout << "Magazine: " << title << ", Issue: " << issueNo << endl;
38- }
39- };
40
41- class DVD : public Media {
42- protected:
43-     string director;
44- public:
45-     DVD(string t, string d) : Media(t), director(d) {}
46- void displayInfo() override {
47-     cout << "DVD: " << title << ", Director: " << director << endl;
48- }
49- };
50
51- int main() {
52-     Book b("Harry Potter", "J.K. Rowling");
53-     Magazine m("Jung", 12345);
54-     DVD d("Inception", "Albert");
55
56-     b.displayInfo();
57-     b.borrowMedia();
58
59-     m.displayInfo();
60-     d.displayInfo();
61-     d.borrowMedia();
62-     d.returnMedia();
63
64-     return 0;
65- }
```

Output

Book: Harry Potter, Author: J.K. Rowling
Borrowed: Harry Potter
Magazine: Jung, Issue: 12345
DVD: Inception, Director: Albert
Borrowed: Inception
Returned: Inception

=== Code Execution Successful ===

LAB 9

EXAMPLE 1

```
1 #include <iostream>
2 using namespace std;
3
4 class Base {
5     int x;
6 public:
7     // Pure virtual function makes this an abstract base class
8     virtual void fun() = 0;
9     int getX() { return x; }
10 };
11
12 // This class inherits from Base and implements fun()
13 class Derived : public Base {
14     int y;
15 public:
16     void fun() {
17         cout << "fun() called" << endl;
18     }
19 };
20
21 int main() {
22     Derived d;
23     d.fun();
24     return 0;
25 }
```

fun() called

=== Code Execution Successful ===

EXAMPLE 2

```
1 #include <iostream>
2 using namespace std;
3 class Base {
4 protected:
5     int x;
6     virtual void fun() = 0; // Pure virtual function makes this an
7     abstract class
8 public:
9     Base(int i) {
10         x = i;
11     }
12 };
13 class Derived : public Base {
14     int y;
15 public:
16     // Calling the Base class constructor using an initializer list
17     Derived(int i, int j) : Base(i) {
18         y = j;
19     }
20     void fun() {
21         cout << "x = " << x << ", y = " << y;
22     }
23 };
24 int main(void) {
25     Derived d(4, 5);
26     d.fun();
27     return 0;
28 }
```

x = 4, y = 5

=== Code Execution Successful ===

EXAMPLE 3

```
1 #include <iostream>
2 using namespace std;
3 class Shape {
4     protected:
5         float dimension;
6     public:
7         void getDimension() {
8             cin >> dimension;    }
9
10        virtual float calculateArea() = 0;
11 };
12 class Square : public Shape {
13     public:
14         float calculateArea() {
15             return dimension * dimension;    }
16 };
17 class Circle : public Shape {
18     public:
19         float calculateArea() {
20             return 3.14 * dimension * dimension;    }
21 };
22 int main() {
23     Square square;
24     Circle circle;
25     cout << "Enter the length of the square: ";
26     square.getDimension();
27     cout << "Area of square: " << square.calculateArea() << endl;
28     cout << "\nEnter radius of the circle: ";
29     circle.getDimension();
30     cout << "Area of circle: " << circle.calculateArea() << endl;
31
32     return 0; }
```

Enter the length of the square: 3
Area of square: 9

Enter radius of the circle: 4
Area of circle: 50.24

=== Code Execution Successful ===

EXERCISE Q1

```
1 //Q1
2 #include <iostream>
3 using namespace std;
4
5 class ArrayMultiplier {
6 public:
7     virtual void calculate() = 0;
8     virtual ~ArrayMultiplier() {}
9 };
10
11 class ArrayMultiplier1D : public ArrayMultiplier {
12     int arr[5] = {1,3,5,7,9};
13 public:
14     void calculate() override {
15         int m;
16         cout<<"Enter number : \n";
17         cin>>m;
18         cout << "1D Array x " << m << endl;
19         for(int i=0; i<5; i++) {
20             cout << arr[i] * m << " ";
21         }
22         cout << endl;
23     }
24 };
25
26 class ArrayMultiplier2D : public ArrayMultiplier {
```

```
Enter number :
4
1D Array x 4
4 12 20 28 36
Enter number :
2
2D Array Multiplied x 2
2 4 6
8 10 12
14 16 18

=== Code Execution Successful ===
```

```
22     cout << endl;
23 }
24 };
25
26 class ArrayMultiplier2D : public ArrayMultiplier {
27     int arr[3][3] = {{1, 2,3}, {4,5, 6}, {7,8,9}};
28 public:
29     void calculate() override {
30         int m ;
31         cout<<"Enter number : \n";
32         cin>>m;
33         cout << "2D Array Multiplied x " << m << endl;
34         for(int i=0; i<3; i++) {
35             for(int j=0; j<3; j++) {
36                 cout << arr[i][j] * m << " ";
37             }
38             cout << endl;
39         }
40
41 int main() {
42     ArrayMultiplier1D a;
43     ArrayMultiplier2D b;
44     a.calculate();
45     b.calculate();
46     return 0;
47 }
```

```
Enter number :
4
1D Array x 4
4 12 20 28 36
Enter number :
2
2D Array Multiplied x 2
2 4 6
8 10 12
14 16 18

=== Code Execution Successful ===
```

Q2

main.cpp	Output
<pre> 1 //Q2 2 #include <iostream> 3 using namespace std; 4 5 class Store { 6 protected: 7 float total_bill; 8 public: 9 Store(float b) : total_bill(b) {} 10 virtual void calculateFinalBill() = 0; 11 virtual ~Store() {} 12 }; 13 14 class ImtiazStore : public Store { 15 public: 16 ImtiazStore(float bill) : Store(bill) {} 17 void calculateFinalBill() { 18 float final_Bill = total_bill - (total_bill * 0.07); 19 cout << "Imtiaz Store Final Bill (7% discount) : " << final_Bill 20 << endl; 21 } 22 }; 23 class BinHashimStore : public Store { 24 public: 25 BinHashimStore(float bill) : Store(bill) {} </pre>	<pre> Enter total bill amount : 40000 Imtiaz Store Final Bill (7% discount) : 37200 Bin Hashim Store Final Bill (5% discount) : 38000 === Code Execution Successful === </pre>

<pre> 19 cout << "Imtiaz Store Final Bill (7% discount) : " << final_Bill 20 << endl; 21 }; 22 23 class BinHashimStore : public Store { 24 public: 25 BinHashimStore(float bill) : Store(bill) {} 26 void calculateFinalBill() { 27 float final_Bill = total_bill - (total_bill * 0.05); 28 cout << "\nBin Hashim Store Final Bill (5% discount) : " << 29 final_Bill << endl; 30 }; 31 32 int main() { 33 float amount; 34 cout << "Enter total bill amount : "; 35 cin >> amount; 36 ImtiazStore Imtiaz(amount); 37 BinHashimStore BinHashim(amount); 38 Imtiaz.calculateFinalBill(); 39 BinHashim.calculateFinalBill(); 40 41 return 0; 42 } </pre>	<pre> Enter total bill amount : 40000 Imtiaz Store Final Bill (7% discount) : 37200 Bin Hashim Store Final Bill (5% discount) : 38000 === Code Execution Successful === </pre>
---	--

Q3

```
1 //q3
2 #include <iostream>
3 #include <cstring>
4 #include <vector>
5 using namespace std;
6 class Vehicle {
7 protected:
8     int carId;
9     string brand, model;
10 public:
11     Vehicle(int id, string b, string m)
12         : carId(id), brand(b), model(m) {}
13     virtual bool isAvailable() = 0;
14     virtual void rentV() = 0;
15     virtual void returnV() = 0;
16     virtual void display() {
17         cout << "ID:" << carId << endl;
18         cout << "brand : " << brand << endl;
19         cout << "model : " << model << endl;
20     }
21     virtual ~Vehicle() {}
22 };
23 class Car : public Vehicle {
24     bool available;
25 public:
26     Car(int id, string b, string m) : Vehicle(id, b, m), available(true)
27     {}
28     bool isAvailable() override { return available; }
29     void rentV() override { available = false; }
30     void returnV() override { available = true; }
31 };
32 class RentalSystem {
33 public:
```

Ali wants to rent : ID:1234
brand : COrolla
model : Model XYZ

Successfully rented:
ID:1234
brand : COrolla
model : Model XYZ

Elsa wants to rent : ID:1234
brand : COrolla
model : Model XYZ

Vehicle
ID:1234
brand : COrolla
model : Model XYZ
is not available.
Elsa wants to rent : ID:5678
brand : Tesla
model : Model PQR

Successfully rented:
ID:5678
brand : Tesla
model : Model PQR

Ali is returning : ID:1234
brand : COrolla
model : Model XYZ

System: ID:1234
brand : COrolla

```

57     cout << endl;
58     rs.rentV(v);
59 }
60 void returnV(Vehicle *v) {
61     cout << name << " is returning : ";
62     v->display();
63     cout << endl;
64     rs.returnV(v); }
65 };
66 int main()
67 {
68     const int s=2;
69     Vehicle *size[s];
70     size[0] = new Car(1234, "C0rolla", "Model XYZ");
71     size[1] = new Car(5678, "Tesla", "Model PQR");
72
73     Customer a("Ali") ;
74     Customer b("Elsa") ;
75
76     a.rentV(size[0]);
77     b.rentV(size[0]);
78     b.rentV(size[1]);
79
80     a.returnV(size[0]);
81     b.rentV(size[1]);
82
83     for (int i = 0; i < s; i++) {
84         delete size[i];
85     }
86     return 0;
87 }
88
89
90

```

```

Vehicle
ID:1234
brand : C0rolla
model : Model XYZ
is not available.
Elsa wants to rent : ID:5678
brand : Tesla
model : Model PQR

Successfully rented:
ID:5678
brand : Tesla
model : Model PQR

Ali is returning : ID:1234
brand : C0rolla
model : Model XYZ

System: ID:1234
brand : C0rolla
model : Model XYZ
has been returned.
Elsa wants to rent : ID:5678
brand : Tesla
model : Model PQR

Vehicle
ID:5678
brand : Tesla
model : Model PQR
is not available.

```

=== Code Execution Successful ===

```

32 public:
33 void rentV(Vehicle *v) {
34     if (v->isAvailable()) {
35         v->rentV();
36         cout << "Successfully rented: " << endl;
37         v->display() ;
38         cout << endl;
39     } else {
40         cout << "Vehicle " << endl;
41         v->display();
42         cout << " is not available." << endl;} }
43 void returnV(Vehicle *v) {
44     v->returnV();
45     cout << "System: ";
46     v->display();
47     cout << " has been returned." << endl; }
48 };
49 class Customer {
50     string name;
51     RentalSystem rs;
52 public:
53     Customer( string n) : name(n) {}
54 void rentV(Vehicle *v) {
55     cout << name << " wants to rent : ";
56     v->display();
57     cout << endl;
58     rs.rentV(v);
59 }
60 void returnV(Vehicle *v) {
61     cout << name << " is returning : ";
62     v->display();
63     cout << endl;
64     rs.returnV(v); }

```

```

Ali wants to rent : ID:1234
brand : C0rolla
model : Model XYZ

Successfully rented:
ID:1234
brand : C0rolla
model : Model XYZ

Elsa wants to rent : ID:1234
brand : C0rolla
model : Model XYZ

Vehicle
ID:1234
brand : C0rolla
model : Model XYZ
is not available.
Elsa wants to rent : ID:5678
brand : Tesla
model : Model PQR

Successfully rented:
ID:5678
brand : Tesla
model : Model PQR

Ali is returning : ID:1234
brand : C0rolla
model : Model XYZ

System: ID:1234
brand : C0rolla

```

Q4

main.cpp

Share

Run

Output

```

1 //q4
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5
6 class EncryptionTechnique {
7 public:
8     virtual string encrypt(string msg) = 0;
9     virtual ~EncryptionTechnique() {} };
10 class EncryptionTechnique1 : public EncryptionTechnique {
11 public:
12     string encrypt(const string msg) override{
13         string result = "";
14         for (char c : msg) {
15             result += to_string((int)c);
16         }
17         return result;
18     }
19 };
20 class EncryptionTechnique2 : public EncryptionTechnique {
21 public:
22     string encrypt(const string msg) override{
23         string result = "";
24         for (char c : msg) {
25             result += to_string((int)c + 2);
26         }
27         return result;
28     }
29 };
30 int main() {
31     string msg;
32     cout << "Enter message to encrypt: ";
33     getline(cin, msg);
34     EncryptionTechnique* t1 = new EncryptionTechnique1();
35     EncryptionTechnique* t2 = new EncryptionTechnique2();
36
37     cout << "Original Message      : " << msg << endl;
38     cout << "EncryptionTechnique1 : " << t1->encrypt(msg) << endl;
39     cout << "EncryptionTechnique2 : " << t2->encrypt(msg) << endl;
40
41     delete t1;
42     delete t2;
43
44     return 0;
45 }

```

Enter message to encrypt: hello guys

Original Message : hello guys

EncryptionTechnique1 : 10410110810811132103117121115

EncryptionTechnique2 : 10610311011011334105119123117

=== Code Execution Successful ===

Enter message to encrypt: hello guys

Original Message : hello guys

EncryptionTechnique1 : 10410110810811132103117121115

EncryptionTechnique2 : 10610311011011334105119123117

=== Code Execution Successful ===